

Recitation 4: Trees and Lists

For the questions on trees, assume each node is a data structure like the following:

```
class Node:
    val: Int
    left: Null // or Node
    right: Null // or Node
```

For linked lists, you may assume a node type

```
class ListNode:
    val: Int
    next: Null // or ListNode
```

1. In class you saw an iterative algorithm for depth-first search. Design a *recursive* algorithm for depth-first search (you may assume preorder: visit the node, then its left subtree, then its right subtree).

Solution. A simple recursive DFS (preorder) starting from a node **root**:

Algorithm 1 RecursiveDFS(**root**)

```
1: if root = Null then
2:   return
3: end if
4: visit(root.val)                                ▷ e.g., print the value
5: RecursiveDFS(root.left)
6: RecursiveDFS(root.right)
```

Correctness:

- Base case: if the tree is empty (**root** = Null), there is nothing to visit.
- Inductive step: assuming the algorithm correctly visits all nodes in the left and right subtrees, the call on **root** first visits **root**, then recursively visits every node in the left subtree, then every node in the right subtree. Every node in the tree is in exactly one of these three sets, so all nodes are visited exactly once.

Running time is $O(n)$ where n is the number of nodes (each node is visited once).

2. A level-order traversal of a tree visits each node of a given level before descending to the next level. Write an algorithm to print all the nodes of a tree in level-order. (Hint: use a queue.)

Solution. We perform a breadth-first search (BFS) using a queue:

Algorithm 2 LevelOrder(root)

```
1: if root = Null then
2:   return
3: end if
4: create an empty queue  $Q$ 
5: enqueue  $root$  into  $Q$ 
6: while  $Q$  is not empty do
7:    $node \leftarrow$  dequeue from  $Q$ 
8:   visit( $node.val$ ) ▷ e.g., print the value
9:   if  $node.left \neq$  Null then
10:    enqueue  $node.left$  into  $Q$ 
11:   end if
12:   if  $node.right \neq$  Null then
13:    enqueue  $node.right$  into  $Q$ 
14:   end if
15: end while
```

Each node is enqueued and dequeued exactly once, and we do $O(1)$ work per node, so the running time is $O(n)$ and the extra space is $O(n)$ in the worst case (for a very wide level).

3. Write an algorithm that returns the height of a tree (max root-to-leaf path length). Assume the height of an empty tree is 0, and the height of a single-node tree is 1.

Solution. A natural recursive definition: the height is 1 plus the maximum of the heights of the left and right subtrees.

Algorithm 3 Height(root)

```
1: if root = Null then
2:   return 0
3: end if
4:  $h_{left} \leftarrow$  Height(root.left)
5:  $h_{right} \leftarrow$  Height(root.right)
6: return  $1 + \max(h_{left}, h_{right})$ 
```

Correctness follows by structural induction: each subtree's height is computed correctly by the recursive call, and then we add 1 to account for the current root.

Each node's height is computed once, so the runtime is $O(n)$.

4. Given a sorted list of distinct integers $a_1 < a_2 < \dots < a_n$ (for example, the result of an inorder traversal of some binary search tree), write an algorithm to construct a *height-balanced* binary search tree containing exactly these keys.

Solution. We can build a height-balanced BST by always choosing the middle element as the root of the (sub)tree, and recursing on the left and right halves.

Algorithm 4 BuildBalancedBST(A, ℓ, r)

```
1: if  $\ell > r$  then  
2:   return Null  
3: end if  
4:  $m \leftarrow \lfloor (\ell + r)/2 \rfloor$   
5:  $root \leftarrow$  new Node  
6:  $root.val \leftarrow A[m]$   
7:  $root.left \leftarrow$  BuildBalancedBST( $A, \ell, m - 1$ )  
8:  $root.right \leftarrow$  BuildBalancedBST( $A, m + 1, r$ )  
9: return  $root$ 
```

To build the tree from the full list, call BuildBalancedBST($A, 1, n$) (or BuildBalancedBST($A, 0, n-1$) if using 0-based indices).

Why is it a BST? All keys in the left subarray $A[\ell \dots m - 1]$ are $< A[m]$, so they go into the left subtree; all keys in the right subarray $A[m + 1 \dots r]$ are $> A[m]$, so they go into the right subtree. Recursively, this preserves the BST property.

Why is it height-balanced? At each step we split the array roughly in half, so the left and right subtrees have sizes differing by at most 1. This yields height $O(\log n)$.

Running time: Each element becomes a node exactly once and we do $O(1)$ work per node, so the total time is $O(n)$.

5. Given two sorted linked lists L_1 and L_2 (in nondecreasing order), merge them to produce a single sorted linked list containing all the elements of L_1 and L_2 .

Solution. We can merge in one pass using two pointers.

Let p_1 point to the head of L_1 and p_2 point to the head of L_2 . We maintain a dummy head to simplify pointer handling.

Algorithm 5 MergeSortedLists(L_1, L_2)

```
1: dummy  $\leftarrow$  new ListNode
2: tail  $\leftarrow$  dummy
3:  $p_1 \leftarrow$  head of  $L_1$ 
4:  $p_2 \leftarrow$  head of  $L_2$ 
5: while  $p_1 \neq \text{Null}$  and  $p_2 \neq \text{Null}$  do
6:   if  $p_1.\text{val} \leq p_2.\text{val}$  then
7:     tail.next  $\leftarrow p_1$ 
8:      $p_1 \leftarrow p_1.\text{next}$ 
9:   else
10:    tail.next  $\leftarrow p_2$ 
11:     $p_2 \leftarrow p_2.\text{next}$ 
12:   end if
13:   tail  $\leftarrow$  tail.next
14: end while  $\triangleright$  attach the remaining nodes (if any)
15: if  $p_1 \neq \text{Null}$  then
16:   tail.next  $\leftarrow p_1$ 
17: else
18:   tail.next  $\leftarrow p_2$ 
19: end if
20: return dummy.next  $\triangleright$  head of merged list
```

At each step we remove the smaller of the two current nodes and append it to the output. Since each node is processed exactly once, the running time is $O(n_1 + n_2)$ where n_1 and n_2 are the lengths of the two lists.

6. Given an array $A[0 \dots n-1]$ representing a complete binary tree, write an algorithm to check in $O(n)$ time whether it satisfies the *min-heap* property (every node is $\leq$ its children). (For a max-heap, reverse the inequality.)

Solution. In a 0-based array representation, for a node at index i :

$$\text{left child index} = 2i + 1, \quad \text{right child index} = 2i + 2.$$

We must check that for every internal node i , $A[i] \leq A[\text{left}]$ and $A[i] \leq A[\text{right}]$ (when the children exist).

It suffices to iterate over all indices i that have at least one child. The last index that can be a parent is $\lfloor (n-2)/2 \rfloor$.

Algorithm 6 IsMinHeap(A)

```
1:  $n \leftarrow \text{length of } A$ 
2: for  $i = 0$  to  $\lfloor (n - 2)/2 \rfloor$  do
3:    $left \leftarrow 2i + 1$ 
4:    $right \leftarrow 2i + 2$ 
5:   if  $left < n$  and  $A[i] > A[left]$  then
6:     return false
7:   end if
8:   if  $right < n$  and  $A[i] > A[right]$  then
9:     return false
10:  end if
11: end for
12: return true
```

Correctness: The min-heap property is a local property: each node must be $\leq$ its children. If the condition holds for every parent node, then by induction down the tree, the entire structure is a min-heap. Conversely, if any parent violates the property, we immediately return **false**.

Running time: We do a constant amount of work for each index i from 0 to $\lfloor (n - 2)/2 \rfloor$, so the time is $O(n)$. The space usage is $O(1)$ extra.